Project Marrow - Remaining Tier Build Plan

Coder-facing roadmap after Tier 1C

Generated 2026-07-08

Audience: coder implementing the remaining prototype/demo tiers.
Scope: after Tier 1C works in Godot.

Table of Contents

Status Baseline

- Assumed current build: Godot 4.x, GDScript, grey-box prototype.
- Completed: Tier 0 core loop and Tier 1A-C prototype steps.
- Working loop: move -> fight enemies -> enemies drop named bones -> hold E to collect -> press Q to cycle equipped bones -> equipped bone changes stats -> complete bone-gated trials -> open exit.
- Existing important files: scripts/player.gd, scripts/enemy.gd, scripts/bone.gd, scripts/arena_goal_manager.gd, scripts/bone_trial_gate.gd, scripts/exit_portal.gd, scenes/main.tscn, scenes/player.tscn, scenes/enemy.tscn, scenes/bone.tscn, scenes/equipped_bone.tscn.
- Primary rule from here: do not expand content until the core loop is readable, repeatable, and easy to tune.

Global Engineering Rules

- Keep every tier playable. Do not leave the project in a half-working state for more than one task.
- Prefer small scenes and scripts per concept: player, enemy, pickup, trial gate, portal, UI, spawner, save system.
- When data starts repeating, move it into a data table or Resource-like structure before adding more content.
- Every mechanic needs a visible test in the grey-box arena before it is considered done.
- Every new feature needs a short acceptance test: what to press, what should happen, and what would count as a failure.
- Avoid animation, art, RPG progression, inventory screens, shops, lore, and real levels until the Tier 1 vertical slice is fun with ugly shapes.

Recommended Near-Term Refactor

- Before adding many more bones, extract bone definitions from scattered match statements.
- Suggested new file: scripts/bone_database.gd or scripts/bone_defs.gd.
- Each bone definition should include id, display_name, color, slot, move_speed_bonus, attack_range_bonus, attack_damage_bonus, optional tags, and short description.
- Player, pickup, enemy, and trial gate scripts should ask the same source for names/colors/effects.
- Done when: adding a new bone type requires changing one data table and assigning an enemy drop, not editing four different scripts.

Tier 1D - Combat Feel Pass

- Goal: make the fight readable enough that bone bonuses can be felt.
- Build a visible attack indicator. Start with a quick translucent arc, cone, or short-lived Area3D in front of the player.
- Add attack cooldown, for example 0.35-0.6 seconds, so holding or mashing Space does not blur the test.

- Add feedback: enemy hit flash, small knockback or bump, hit sound placeholder, and a short player attack flash.
- Add player facing or attack direction. The current range check is serviceable, but a Zelda-like game eventually needs directional intent.
- Add enemy contact danger only if the fight still feels too empty. If added, keep it simple: touching enemy reduces player HP or pushes player back.
- Suggested files: scripts/player_attack.gd or keep in player.gd for one more pass; scripts/enemy.gd; optional scenes/attack_hitbox.tscn.
- Done when: a new tester can tell when they attacked, whether it hit, which enemy was hit, and why the enemy died.

Tier 1E - Bone System Cleanup

- Goal: turn the prototype bone logic into a small system that can survive more content.
- Create a shared bone definition source. Remove duplicated display names, colors, and stat effects from player.gd, bone.gd, enemy.gd, and bone_trial_gate.gd.
- Support at least one slot explicitly, e.g. right_arm. Keep left_arm, legs, and heavy/body slots as planned but not fully implemented unless needed.
- Define equip replacement rules: equipping another right_arm bone replaces the previous right_arm bone; later slots can coexist.
- Update inventory UI so it shows collected bones and currently equipped slot(s). It can remain text-based.
- Allow duplicate pickup policy to be explicit. Recommended for now: collecting an already-owned bone does not duplicate; it updates or ignores with a message.
- Done when: Arm Bone, Leg Bone, and Heavy Bone are all defined in one place and Q/equip behavior is predictable.

Tier 1F - Grey-Box Vertical Slice

- Goal: one complete 5-minute grey-box arena that proves the game idea has play value.
- Build a simple start -> fight -> harvest -> swap -> solve trials -> exit flow.
- Place enemies and trials so the player naturally learns: gold/arm means reach, green/leg means speed, purple/heavy means power with a speed tradeoff.
- Add a restart/reset key or button after completion so repeated testing is fast.
- Add a basic win state screen or label: Tier 1 complete / time / bones used.
- Add one optional alternate route that rewards experimenting, but do not create a full level yet.
- Done when: someone unfamiliar can play from spawn to exit without developer explanation, using only on-screen labels.

Tier 1G - Playtest And Decision Gate

- Goal: decide whether the core loop is fun enough to justify Tier 2.
- Run at least 5 playtests with people who did not build it.
- Observe without explaining. Track confusion points, unused bones, boring moments, and whether players voluntarily swap bones.

- Questions to answer: Did they understand kill -> harvest -> equip? Did they swap because they wanted to or because the gate forced them? Which bone felt best? Which felt pointless? Did combat feel fair?
- Make a short playtest notes file: docs/tier1_playtest_notes.md.
- Decision gate: continue only if swapping bones creates at least one genuinely interesting choice. If not, redesign bone effects before Tier 2.

Tier 2A - Demo Architecture Pass

- Goal: prepare the codebase for a small playable demo without overengineering.
- Introduce folders if not already present: scripts/player, scripts/enemies, scripts/bones, scripts/ui, scripts/level, scripts/save.
- Separate player movement, combat, inventory/equipment, and stats if player.gd becomes hard to read.
- Introduce signals for major events: bone_collected, bone_equipped, enemy_died, trial_completed, level_completed.
- Replace direct get_tree group calls only where they start hiding bugs. Groups are fine for prototypes; signals are better for demo-scale systems.
- Done when: a coder can add a new enemy, bone, or gate without reading the whole project.

Tier 2B - First Real Demo Level

- Goal: one small real level, still simple, that lasts about 10-15 minutes.
- Layout: spawn area, first enemy/tutorial, branching bone challenges, one climax encounter, exit/goal.
- Use grey-box first. Only add art after the level is playable from beginning to end.
- Use reusable level pieces: floor tiles, walls, ramps, doors/gates, trial props, enemy spawn markers.
- Add a simple camera polish pass so visibility is reliable in the level.
- Done when: a friend can complete the level in one sitting and understand why each bone matters.

Tier 2C - Enemy Types

- Goal: enemies are no longer identical boxes with different drops.
- Add 3 enemy archetypes tied to bone rewards: reach enemy drops Arm Bone, runner enemy drops Leg Bone, bruiser enemy drops Heavy Bone.
- Each enemy should have one readable behavior. Examples: reach enemy keeps distance, runner circles or dashes, bruiser moves slowly but hits harder.
- Keep AI primitive. Use simple states: idle, chase, attack, dead.
- Add clear silhouettes/colors while still grey-boxing.
- Done when: players can guess what a creature might drop from how it looks and behaves.

Tier 2D - Progression And Save/Load

- Goal: make the demo persistent enough to feel like a game.

- Save: collected bones, equipped bones, completed level flags, player position if needed.
- Keep save data small and human-readable if possible, such as a Dictionary serialized to user://save.json.
- Add a reset save option for testing.
- Progression should be minimal: bones unlock paths, not stat menus or XP trees yet.
- Done when: closing and reopening the game preserves bones and completed demo progress.

Tier 2E - UI And Controls Pass

- Goal: make the prototype understandable without Codex explaining it.
- Replace raw text panels with simple but readable HUD elements: current bone, stats/effects, pickup prompt, objective tracker.
- Add a controls panel or pause overlay: WASD move, Space attack, E pickup/inventory, Q equip next bone.
- Make prompts contextual: Hold E to pick up Arm Bone, Press Q to equip next bone, Needs Heavy Bone.
- Done when: a first-time player can discover controls and objectives inside the game.

Tier 2F - Art Direction Starter Pass

- Goal: establish visual consistency without drowning in asset production.
- Pick a simple stylized look: clean shapes, readable colors, strong silhouettes, good lighting.
- Replace only the most important grey boxes first: player, three enemies, three bone pickups, one level kit.
- Avoid mixing mismatched asset packs. Consistency beats fidelity.
- Keep prototype colors meaningful: bone type color should remain readable even after art improves.
- Done when: screenshots look intentionally stylized rather than like unstyled debug shapes.

Tier 2G - Audio And Juice

- Goal: make actions feel satisfying at low cost.
- Add placeholder sounds: attack swing, hit, enemy death, bone drop, pickup, equip, gate complete, exit open.
- Add small particles or simple visual effects for hit, pickup, equip, and gate completion.
- Do not chase cinematic polish. Each effect should clarify game state.
- Done when: the game is understandable with the HUD hidden for a few seconds.

Tier 2H - Demo Packaging

- Goal: make a build someone else can run.
- Create export presets for the target desktop platform first.
- Test a clean export outside the editor.
- Create a short README with controls, known issues, and feedback questions.
- Make a playtest build folder and version names, e.g. MarrowDemo_0_1_0.

- Done when: someone can download/run the build and play the 10-15 minute demo without opening Godot.

Tier 3A - Full Game Planning Gate

- Goal: only plan the full game after Tier 2 proves the loop.
- Use Tier 2 playtest results to choose final scope. Do not assume the current bone list, controls, or combat are final.
- Define production pillars: number of areas, number of enemy families, number of bone slots, target playtime, target platform.
- Create a content budget. Example: 4 areas, 12 enemies, 12 bones, 4 bosses is already ambitious for solo development.
- Done when: the full game plan is based on evidence from the demo, not the original fantasy alone.

Tier 3B - Content Pipeline

- Goal: make adding content boring and reliable.
- New bone pipeline: create bone definition -> pickup visual -> equipped visual -> enemy drop -> optional gate/use case -> test scene.
- New enemy pipeline: create enemy scene -> stats -> behavior script -> drop table -> test encounter.
- New level pipeline: grey-box -> route test -> encounter pass -> bone gates -> art pass -> audio pass -> playtest.
- Create test scenes for isolated systems so every change does not require playing the whole game.
- Done when: content creation follows checklists instead of improvisation.

Tier 3C - Production Areas

- Goal: build the full game’s areas one at a time.
- Each area should introduce one new bone idea, one new enemy behavior, and one traversal/combat wrinkle.
- Finish areas vertically rather than roughing out the whole world. A finished small area teaches more than five empty biomes.
- Keep revisiting the design pillar: your body is your build. If an area does not use bone swapping, redesign or cut it.
- Done when: each area has a start, core encounter loop, bone reward, use case for that bone, and exit/progression beat.

Tier 3D - Bosses And Set Pieces

- Goal: create memorable tests of bone mastery.
- Bosses should reward swapping, not just high stats.
- Example boss pattern: phase requires Arm Bone reach, phase requires Leg Bone dodging, phase requires Heavy Bone damage window.

- Build boss prototypes with boxes first. Add art and animation only after patterns are readable.
- Done when: bosses test the core system instead of ignoring it.

Tier 3E - Polish, Accessibility, And Release

- Goal: prepare for a real public release.
- Polish controls, camera, UI, save safety, settings, input remapping, volume controls, and readable text.
- Accessibility basics: subtitles/text clarity, color-blind-safe cues beyond color, adjustable camera sensitivity if camera control is added.
- Steam/release basics: build automation, crash notes, store capsule art, trailer footage, demo page if needed.
- Done when: the game can be played by strangers repeatedly without you standing nearby to interpret it.

Immediate Next Coding Task

- After Tier 1C works, build Tier 1D: combat feel pass.
- Most valuable first task: replace pure distance attack with a short-lived visible attack area in front of the player.
- Acceptance test: Space creates a visible attack flash; only enemies inside the flash take damage; the enemy HP label updates; misses are visually understandable.
- Do not start Tier 2 art or save/load until Tier 1D-G are playtested.